certora

Security Assessment

Final Report



Zolana

October 2026

Prepared for Helius Labs

certora HELIUS

Table of Contents

Project Summary **4**
- Project Scope 4
- Project Overview 4
- Protocol Overview 4

Assessment Methodology **5**

Threat and Security Overview **6**
- System model and review scope 6
- Expected security invariants 6
- Trust boundaries and security assumptions 7
- Attack vectors considered 7
- Testing and coverage 8
- Highest severity findings and security takeaways 8

Findings Summary 10

Severity Matrix 10

Detailed Findings **11**

High Severity Issues 13
- H-01: Deposits can bypass data authorization 13

Medium Severity Issues 15
- M-01: Excess reserves block rent recovery from active trees 15
- M-02: Unbounded tree fees can block spending or drain the payer's SOL 17
- M-03: Malicious merger can lock the owner's funds 19

Low Severity Issues 21
- L-01: Tree parameters can change during initialization 21
- L-02: Batch update reimbursements are misattributed across forester workers 23
- L-03: Outputs can enter trees with exhausted nullifier capacity 24
- L-04: Stale proofs can replace correct cached tree updates 26
- L-05: Public cache selection can link an output to its spend 28
- L-06: Fee-payer signatures implicitly authorize shielded spends 30

Informational Severity Issues 32
- I-01: UTXO data is not validated against data_hash 32
- I-02: Ring deposits can drain SPL accounts controlled by ring_auth 33
- I-03: Ring authority can withdraw through public settlement 35
- I-04: Sha256BE returns incorrect zero nodes 36
- I-05: P256 UTXOs have limited support outside custom rings 37
- I-06: Wrong error variant in UtxoTreeLayout::init 38
- I-07: HashChain4 does not bind input length 39
- I-08: Nullifier PDAs lack a type discriminator 40

Disclaimer **41**

About Certora **41**

certora

HELIUS

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Zolana	https://github.com/helius-labs/zolana	0125d8a (initial) bbdecee (final)	Solana, ZK

Project Overview

This document describes the security review of **Zolana**. The work was undertaken from **September 7th, 2026** to **October 13th, 2026**.

The following contract list is included in our scope:

programs/shielded-pool/src/*
program-libs/tree/src/*
prover/server/circuits/gadget/*
prover/server/circuits/ring-utils/*
prover/server/circuits/spp_merge/*
prover/server/circuits/spp_transaction/*

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following pages.

Protocol Overview

Zolana is a privacy protocol on Solana that supports deposits, confidential transfers, and withdrawals of SOL and supported SPL tokens. Balances consist of UTXOs, individual records of spendable value backed by assets held in the Shielded Pool Program. Zero-knowledge proofs validate transfers without exposing private amounts and assets, while the program checks authorization and prevents double spending.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

certora HELIUS

Threat and Security Overview

System model and review scope

Zolana implements confidential asset transfers on Solana using unspent transaction outputs (UTXOs), or notes. The Shielded Pool Program holds SOL and token collateral, records note commitments, verifies zero-knowledge proofs, and records nullifiers that prevent repeated spending. Deposits move public assets into custody and create notes; transfers consume notes and create new ones; withdrawals release collateral; merges consolidate notes. Authorized foresters maintain the nullifier tree through proved batch updates and reclaim temporary account rent.

The manual review focused on the shielded pool program (SPP), tree library, and transaction, merge, and nullifier-update circuits. Reviewed flows included deposits, public settlement, multi-tree transactions, cached spends, delegated merges, tree initialization, administrative controls, fee collection, batch processing, and nullifier-account cleanup. SDKs, the user registry, indexer, and custom ring programs supplied integration context but they're technically out of scope for this review.

Expected security invariants

The review examined the following intended invariants:

- **Value conservation and collateral**: Value is conserved separately for each asset, and newly created notes are backed by deposited collateral or consumed notes.
- **Authorization**: Spending requires the applicable owner, ring, or delegated authorization. Application-bearing notes must satisfy their owner-authorization requirements.
- **Double-spend prevention**: Real inputs prove commitment inclusion, and nullifiers are correctly derived and distinct. Historical roots must remain usable without allowing repeated spending.
- **Safe maintenance**: Temporary nullifier accounts remain open until cleanup is safe. Queue transitions, batch updates, and capacity reservations preserve spendability.
- **Merge recovery**: Merges preserve ownership, asset, and value while leaving the resulting note recoverable by its owner.

- **Proof and execution consistency**: Amount ranges, field encodings, tree routing, and public-input hashes agree across Rust and Go. Failed transactions roll back state changes and settlement.

Trust boundaries and security assumptions

Depositors authorize funding, while note owners authorize ordinary spending through Solana signatures or in-circuit P256 verification. Custom rings authorize cross-program calls and impose additional policy restrictions. An enabled ring-authority path can replace owner signatures, subject to ring authorization and knowledge of the required note secrets. Delegated merging relies on registry opt-in and preserves ownership without requiring a signature for each merge.

Protocol authorities control configuration, admission, and maintenance permissions; fee authorities control tree charges and reserve recovery. These privileges create distinct availability and economic risks.

Security assumes correct Solana execution and transaction atomicity, sound cryptographic primitives, secure Groth16 setup, and matching circuit and proving/verifying-key artifacts. Token custody depends on validated SPL Token and Token-2022 accounts and supported extension behavior.

Indexer data is checked through accepted roots and proofs, but indexer availability affects wallet operation. A hosted prover receives private witnesses and therefore introduces a confidentiality boundary. Applications must validate payloads against committed hashes, and ring programs must enforce their intended restrictions on public settlement and signer privileges.

Privacy depends on the transaction path. Public deposits and withdrawals reveal amounts and assets, ordinary signing identities remain visible, and public cache selection can reveal additional links between outputs and spends.

Attack vectors considered

The threat model included malicious transaction submitters and depositors, abusive or compromised authorities, malicious delegated mergers, and unsafe application or ring integrations.

The review considered unauthorized note creation, forged application state, replay and double spending, malformed proofs and encodings, account substitution, cross-program signer escalation, stale maintenance updates, premature cleanup, and capacity exhaustion.

Economic review covered unexpected fee increases, payer SOL exposure, reimbursement attribution, and reserve accounting. Delegated merges were examined for owner recovery failures, while ring integrations were examined for unintended withdrawal capabilities and P256 spending limitations.

Testing and coverage

Existing test coverage includes successful and rejected deposits, transfers, withdrawals, merges, administrative changes, proof tampering, nullifier replay, queue transitions, settlement, and rollback.

Recorded review validation exercised compilation of the 8-input and 36-input merge circuits, valid-witness checks for the 8-input shape, recovery derivation vectors, and witness serialization. Property-based deposit lifecycle and input-mutation tests are present, but the available records do not establish a dedicated audit fuzzing campaign or execution of every available suite.

Documented gaps include exhaustive public-input tampering, some exact merge state and event assertions, and portions of full-batch maintenance coverage. No formal verification was performed during this audit.

Highest severity findings and security takeaways

The highest severity findings were:

- **Deposit authorization bypass**: Proofless deposits could create application-bearing notes for another owner without that owner's authorization.
- **Excess rent reserves**: Reserve calculations counted funding already held by live nullifier accounts, preventing surplus recovery.
- **Unbounded tree fees**: Fee changes could block spending or consume more payer SOL than expected.
- **Delegated merge griefing**: A malicious merger could combine known notes with an unrecoverable note, making previously spendable funds inaccessible.

These findings emphasize consistent authorization across note-creation paths, reliable owner recovery, payer-approved fee limits, and accurate accounting for maintenance capital.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	1	1	1
Medium	3	3	3
Low	6	6	3
Informational	8	8	4
Total	18	18	11

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

certora HELIUS

Detailed Findings

ID	Title	Severity	Status
H-01	Deposits can bypass data authorization	High	Fixed
M-01	Excess reserves block rent recovery from active trees	Medium	Fixed
M-02	Unbounded tree fees can block spending or drain the payer's SOL	Medium	Fixed
M-03	Malicious merger can lock the owner's funds	Medium	Fixed
L-01	Tree parameters can change during initialization	Low	Fixed
L-02	Batch update reimbursements are misattributed across forester workers	Low	Acknowledged
L-03	Outputs can enter trees with exhausted nullifier capacity	Low	Fixed
L-04	Stale proofs can replace correct cached tree updates	Low	Acknowledged
L-05	Public cache selection can link an output to its spend	Low	Acknowledged
L-06	Fee-payer signatures implicitly authorize shielded spends	Low	Fixed

I-01	UTXO data is not validated against data_hash	Informational	Acknowledged
I-02	Ring deposits can drain SPL accounts controlled by ring_auth	Informational	Fixed
I-03	Ring authority can withdraw through public settlement	Informational	Acknowledged
I-04	Sha256BE returns incorrect zero nodes	Informational	Fixed
I-05	P256 UTXOs have limited support outside custom rings	Informational	Acknowledged
I-06	Wrong error variant in UtxoTreeLayout::init	Informational	Fixed
I-07	HashChain4 does not bind input length	Informational	Acknowledged
I-08	Nullifier PDAs lack a type discriminator	Informational	Fixed

High Severity Issues

H-01: Deposits can bypass data authorization

Severity: High	Impact: High	Likelihood: High
Files: programs/shielded-pool/src/instructions/deposit/processor.rs	Status: Fixed	

Description:

The shielded pool's proofless deposit instructions create UTXOs directly from caller-supplied instruction data. Each UTXO includes a `data_hash` that commits to application data. In the owner-signed `Transact` path, ConstrainOutput requires the owner to participate whenever a real output carries a nonzero `data_hash`. This prevents callers from creating application state on behalf of another owner without authorization.

However, deposit account validation requires the depositor's signature and, for `ring_deposit`, the `ring_config` signature, without binding the note's owner to a signer. The shared processor accepts `utxo_data.data_hash` for deposit and `data_hash` for `ring_deposit`, checks their field encoding, and incorporates them directly into the UTXO commitment. A depositor can therefore create a note for another owner with an arbitrary nonzero application-data hash without that owner's participation. The ring path similarly lacks an owner-authorization check despite requiring ring approval.

The resulting note has a valid commitment and can pass inclusion checks, but its committed data was never authorized by its owner. Applications and ring policies that rely on the owner-authorization invariant can accept forged application state and make decisions using attacker-selected data. Depending on how the application interprets `data_hash`, this can bypass authorization rules or policy limits.

Recommendations:

Remove application data from proofless deposits and always commit a zero `data_hash` in the shared deposit processor. Require data-bearing UTXOs to be created through proven transaction paths that enforce the applicable owner-authorization constraints.

Customer Response:

Fixed in 1305a42.

Fix Review:

Fix confirmed. Both proofless deposit rails now commit a zero application-data hash, so deposits cannot bypass application-data authorization by supplying their own hash.

Medium Severity Issues

M-01: Excess reserves block rent recovery from active trees

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: programs/shielded-pool/src/instructions/claim_tree_lamports.rs	Status: Fixed	

Description:

`process_claim_tree_lamports` lets the fee authority recover SOL above a tree's reserve. Rent is the minimum balance that exempts an account from storage charges. A nullifier identifies a spend and prevents repeated spending. A program-derived account (PDA) stores each nullifier until cleanup is safe. The tree funds these PDAs and receives their balances when cleanup closes them.

The claim instruction reserves the tree's own rent, `fee_balance` owed for maintenance, and three complete batches of PDA funding at the current rent. This calculation requires all three batches of funding to remain in the tree account. It ignores funding already held by live PDAs, so it counts that portion of the operating capital twice.

Cleanup cannot remove this duplication. A PDA is closable only when its queue_index is below close_before_index. Completing a batch advances close_before_index to that batch's start index, so closing its PDAs requires completing a successor batch. The successor's PDAs must then remain deployed.

For example, ignore the tree's own rent and maintenance fees:

1. A tree receives 30 SOL to fund three PDA batches at 10 SOL per batch.
2. Creating one batch transfers 10 SOL to its PDAs and leaves 20 SOL in the tree.
3. A 10% rent decrease lowers the claim instruction's required liquid reserve to 27 SOL. The tree holds only 20 SOL, so the instruction returns `NoClaimableTreeLamports`.
4. Two further batches now require only 18 SOL. The remaining 2 SOL in the tree exceeds that funding requirement but cannot be claimed.

Part or all of the surplus from a rent decrease remains unavailable to the fee authority. Closing older PDAs cannot eliminate the excess reserve because the newest batch must remain deployed. The current program therefore prevents full recovery even after all eligible cleanup completes.

Recommendations:

Account for funding held by live PDAs when calculating the liquid reserve. Track PDA creation and closure. Use current rent requirements and cleanup timing to calculate the additional liquid funding needed for continued operation. Reserve only that additional funding, the tree's own rent minimum, and `fee_balance`.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. The tree now counts open nullifier PDAs, and the claim reserve funds only the additional PDAs that could still be created, plus the tree account's own rent and its maintenance fee balance.

M-02: Unbounded tree fees can block spending or drain the payer's SOL

Severity: Medium	Impact: High	Likelihood: Low
Files: programs/shielded-pool/src/instructions/set_tree_fees.rs	Status: Fixed	

Description:

A nullifier identifies a spend and prevents repeated spending. Spending instructions charge the payer `fee_per_nullifier` for each nullifier and collect the fees in the tree account. TreeAccount::init stores the `TreeFeeSchedule` from `process_create_tree` without an insertion-fee cap. process_set_tree_fees also applies uncapped fees immediately through `TreeAccount::set_fee_schedule`. Both paths accept rates up to the integer type limit.

The supplied CLI initialization disables permissionless creation, which otherwise lets anyone create a tree. With this setting, only the configured tree creation authority can choose initial fees. Users can review initial fees before depositing, but this does not protect them against later increases.

Authorities can manage fees, but these permissions must not let them seize users' funds through charges that users did not approve. A malicious or compromised fee authority can still raise fees after the payer signs and before execution. `TransactIxData` contains no maximum fee that the payer approves, so execution can collect a higher fee than expected.

Deposits accept trees with excessive fees, but later spending can fail because of unaffordable charges or overflow in fee calculations. If the transaction succeeds, it can consume most or all of the payer's available SOL outside the shielded pool. Taking funds beyond the payer's expected fee exceeds the authorities' intended permissions and justifies Medium severity. Lower fees can restore spending, and failed transactions reverse any forester fee transfer. A forester maintains the nullifier tree and receives collected fees through maintenance payments. An attacker needs control of, or cooperation from, the forester authority to recover these fees.

Recommendations:

Use a shared fee validation function in `TreeAccount::init` and `TreeAccount::set_fee_schedule` to enforce a maximum `fee_per_nullifier` value.

Optionally, also add a maximum total forester fee to the instruction data that the payer signs. Before collecting fees, require the total across all input trees to stay within that signed limit. Apply this protection to all instructions that collect nullifier fees.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. Tree creation and fee updates now cap `fee_per_nullifier` at 1,000,000 lamports (0.001 SOL), while documentation acknowledges that transactions pay the execution-time rate.

M-03: Malicious merger can lock the owner's funds

Severity: Medium	Impact: High	Likelihood: Low
Files: prover/server/circuits/spp_merge/shared/transaction.go	Status: Fixed	

Description:

Owners can enable delegated merging and provide a merger with their UTXO openings and `nullifier_secret` to generate merge proofs. Once merging is enabled, merge_transact accepts merges without the owner's signature. The merge circuit preserves the common owner and asset, sums the input amounts, and derives the output blinding deterministically.

However, the circuit exposes no verifiably encrypted merged amount. Instead, wallet reconstruction requires every real input to be known and sums their amounts locally. This assumes that the owner knows all merged inputs, which the protocol does not enforce. A sender can create a valid UTXO for another owner through `transact` without publishing ciphertext that correctly reveals its amount.

A malicious delegated merger can trigger the issue as follows:

1. Create a UTXO for the owner through `transact`, using the same asset as the owner's known UTXOs, and withhold the correct ciphertext needed to recover its amount.
2. Merge this unknown UTXO together with the owner's known UTXOs. The merger knows all input openings and has the delegated `nullifier_secret`, so the merge satisfies the ownership and value constraints.
3. The owner's known inputs are consumed, but the owner cannot reconstruct the merged amount because one input amount remains unknown.

The owner cannot generate a valid spending proof for the merged UTXO without its exact amount. Consequently, a malicious merger can make previously spendable funds inaccessible until the missing amount is disclosed or otherwise recovered. The output still belongs to the owner, but deterministic blinding alone does not guarantee that the owner can spend it.

Recommendations:

1. Restore verifiable encryption of the merged output to the owner's registered viewing key, as removed in PR #171. Bind the correct ciphertext and registered key to the merge proof and publish the ciphertext so recovery does not depend on knowing every input.

2. Alternatively, constrain an encrypted amount in the merge circuit:

   ```
   enc_amount = merged_amount + Poseidon(ENC_AMOUNT_DOMAIN, nullifier_secret, Nullifiers[0])
   ```

   Use arithmetic in the circuit's field, a distinct encryption domain, and include `enc_amount` in the verified public-input hash and published instruction data. The owner can subtract the same mask to recover `merged_amount`. The first input is constrained to be real, so its single-use nullifier prevents mask reuse across successful merges. Update wallet recovery and the proving and verifying keys accordingly.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. Now merges on the default ring binds encrypted output amount and mint information to the proof, allowing owners to recover notes without every input opening. For merges in custom rings, the ciphertext remains unverified; this has been acknowledged by Helius.

Low Severity Issues

L-01: Tree parameters can change during initialization

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/shielded-pool/src/instructions/create_tree/processor.rs	Status: Fixed	

Description:

`process_create_tree` creates a tree account through four allocation steps. The first step collects funds based on its nullifier parameters, which control how the tree records spent values. It does not store those parameters, the fees, or the initiator. The final step uses the values supplied to TreeAccount::init.

If creation spans separate transactions, another permitted caller can complete the tree with different parameters or fees. With `tree_creation_is_permissionless` enabled, any signer can complete it. Otherwise, the caller must use the configured `tree_creation_authority`. The standard SDK builds identical steps, and the normal creation flow sends them in one transaction to prevent this interference.

If the final batch size requires less funding, fund_tree leaves the excess in the tree without refunding the original payer. Only the fee authority can recover it through claim_tree_lamports after initialization. Existing instructions cannot change the final nullifier parameters, although the fee authority can update fees through set_tree_fees.

Recommendations:

Store the intended nullifier parameters and fees during the first allocation step. Require every later step to match these values before it changes the tree or transfers funds. If the initiator must control completion, also store a designated initialization authority. Require that authority to sign subsequent steps.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. Tree creation now records the first step's payer and exact parameters/fees, requiring every continuation to match, and validating parameters before allocation or tree-ID consumption.

L-02: Batch update reimbursements are misattributed across forester workers

Severity: Low	Impact: Medium	Likelihood: Low
Files: programs/shielded-pool/src/instructions/batch_update_nullifier_tree.rs	Status: Acknowledged	

Description:

`process_batch_update_nullifier_tree` accepts authorized forester proofs, caches updates that arrive out of order, and applies consecutive updates when their predecessors become available. A submission that only caches an update receives no reimbursement. However, CachedTreeUpdate stores only the old root, new root, and occupancy flag, so it does not retain the submitter's reimbursement recipient.

When a later submission triggers application, event.num_update counts every update in the cascade, including proofs previously submitted by other workers. The processor calculates reimbursement for this total and pays it entirely to the current instruction's `reimbursement_recipient`.

`ProtocolConfig` currently stores a single `forester_authority` account, but this account is intended to be a 1-of-N Squads smart account. Different members can submit through the same authority while using separate fee payers and reimbursement recipients. Independently funded workers can therefore incur transaction fees while the member whose submission triggers application receives the reimbursement attributable to their cached updates.

Recommendations:

Record the reimbursement recipient for each accepted cached update and credit that recipient when the update is applied, through supplied writable accounts or a recipient-bound claim balance. Alternatively, acknowledge this issue and document it so that foresters are aware of it.

Customer Response:

Acknowledged in bbdecee.

L-03: Outputs can enter trees with exhausted nullifier capacity

Severity: Low	Impact: High	Likelihood: Low
Files: program-libs/tree/src/lib.rs	Status: Fixed	

Description:

Each tree account contains a UTXO tree for output commitments and a nullifier tree that records spends. Deposit, transact, and merge append outputs based on UTXO capacity without verifying that the destination retains nullifier capacity to spend those outputs.

The `allow_dummy_inputs` check compares remaining nullifier slots against unused UTXO leaves before queuing current inputs. This omits existing unspent notes from the reserve and does not account for the current batch's reservations. Dummy/address inputs consume nullifier slots without spending existing notes, while cross-tree transactions insert nullifiers into input trees separately from destination outputs. The nullifier tree can therefore become full while the UTXO tree still has room.

Users can then deposit or receive funds that become permanently unspendable because every later spend fails with TreeIsFull. Existing unspent notes in that tree are also frozen. A replacement tree cannot recover them, since moving notes requires inserting nullifiers into the exhausted source tree. The impact is limited to the affected tree and requires approximately `2^40` lifetime nullifier reservations, making deliberate exhaustion extremely costly.

Recommendations:

Reserve nullifier slots for the UTXO tree's total capacity, rather than only its unused leaves. Replace `allow_dummy_inputs` with `dummy_input_headroom`, computed as `remaining_queue_capacity` minus total UTXO capacity, clamped at zero. Before inserting nullifiers, transact and merge should permit dummy/address inputs only when this headroom covers the full current input count for that tree.

Customer Response:

Fixed in ebca3ad.

Fix Review:

Fix confirmed. The new code reserves nullifier capacity for the full UTXO tree and permits dummy/address inputs only when each source tree has headroom for the entire input group.

L-04: Stale proofs can replace correct cached tree updates

Severity: Low	Impact: Low	Likelihood: Low
Files: program-libs/tree/src/nullifier_tree/merkle_tree_update.rs	Status: Acknowledged	

Description:

Authorized foresters update the nullifier tree, which records spent notes, through `update_tree_from_queue`. The program caches verified updates for future queue positions. Proof verification binds the queued values and queue position, but does not establish that the starting root follows earlier queued updates. `cache_tree_update` then replaces an occupied position without comparing its existing roots.

Although `ProtocolConfig` stores only one `forester_authority` account, the specification intends it to be a 1-of-N Squads smart account. Each authorized member can independently submit through that account, so a malicious member can interfere with honest members' updates. A valid proof from an outdated tree root can therefore discard an honest forester's cached update:

1. An honest forester caches a correct update for a future queue position.
2. Another authorized forester submits a valid proof for that position from an outdated root, replacing the correct update.
3. When processing reaches that position, apply_cached_tree_updates detects the root mismatch, evicts the update, and stops. Later cached updates remain unapplied.

A malicious authorized forester can delay tree updates and force honest workers to resubmit proofs and pay additional transaction fees. If both queue batches fill, spends from the affected tree fail until processing resumes.

Recommendations:

Reject a submission if its `old_root` or `new_root` conflicts with an occupied cache position, and treat identical resubmissions as no-ops. Preserve the root comparison and stale-update eviction so that workers can recover unusable cache entries.

For stronger protection, accept only the next unapplied queue position and require `old_root` to equal the current tree root before caching. This alternative removes support for updates submitted out of order.

Customer Response:

Acknowledged.

L-05: Public cache selection can link an output to its spend

Severity: Low	Impact: Low	Likelihood: Low
Files: programs/shielded-pool/src/instructions/transact/verify.rs	Status: Acknowledged	

Description:

The UTXO cache stores verified output commitments in a public account, allowing later transactions to spend them without waiting for Merkle inclusion data. A transaction's public `read_bitmap` selects cache slots, and their commitments are bound into the proof's public input hash. Selected entries can remain unused, so selecting a slot alone does not prove that its commitment was spent.

For a successful transaction with one input, one selected cache slot, no UTXO root, and a strictly positive net public withdrawal, the relationship becomes conclusive:

1. Balance conservation requires a real shielded input to fund the withdrawal, since dummy and address inputs carry no value.
2. Omitting the UTXO root supplies a zero root. The input constraints require a nonzero root for an uncached UTXO, forcing this input to use the cache.
3. The cache constraints require the input commitment to equal the selected commitment, and nullifier derivation binds that input to the transaction's sole nullifier.

An observer can therefore identify when the earlier shielded output was spent, weakening its unlinkability. This does not break commitment hiding or recover the owner secret, shielded amount, asset, or recipient from the commitment. Public withdrawal details are already visible independently. The inference depends on the conditions above.

Recommendations:

Document this privacy risk in the cache and SDK guide, including the conditions that make the link conclusive and the fact that selecting a slot alone is insufficient.

Explain that the cache is mainly useful for short-lived operations that process many shielded outputs, such as batches of merges, because it allows dependent proofs to be prepared concurrently without waiting for indexer updates and Merkle inclusion data.

Integrations that need stronger unlinkability should wait for normal tree inclusion data and spend without public cache selection.

Customer Response:

Acknowledged in bbdecee.

L-06: Fee-payer signatures implicitly authorize shielded spends

Severity: Low	Impact: High	Likelihood: Low
Files: programs/shielded-pool/src/instructions/transact/verify.rs	Status: Fixed	

Description:

The shielded pool treats the instruction's payer as an eligible shielded-spend signer. The `fill_owner_signer_hashes` function always inserts the payer into the owner-signer vector, even when no owner-signer accounts are supplied. The Ed25519 input authorization check accepts each private input-owner identity that appears anywhere in this vector, including the payer's identity.

This creates an authorization risk when an account sponsors an externally supplied transaction:

1. The sponsoring account owns shielded UTXOs, and the transaction's author already knows its nullifier secret and sufficient UTXO data to construct a valid proof.
2. The account signs as the instruction payer, intending only to sponsor fees, without approving a spend of its shielded funds.
3. The payer's signature also satisfies the circuit's input-owner authorization. The actual input ownership remains private, so the account roles alone cannot identify whether the payer's UTXOs are consumed.

Under these conditions, a sponsored transaction can spend the payer's shielded funds without a separate owner-signer declaration. The specification explicitly defines payer-first authorization, making this a design risk for integrations that assume payer-only sponsorship.

Recommendations:

Separate fee payment from shielded-spend authorization. Require every Ed25519 input owner to be explicitly declared in the owner-signer accounts, including an owner that also pays.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. The new code now excludes the fee payer from implicit owner authorization and requires paying owners to appear explicitly in owner-signer slots.

Informational Severity Issues

I-01: UTXO data is not validated against data_hash

Files:

prover/server/circuits/spp_transaction/shared/outputs.go

Description:

The Shielded Pool verifies UTXO commitments that include `data_hash`. However, its UTXO circuit fields contain only the supplied hash, without the application-data bytes needed to verify it. The specification explicitly states that `data_hash` is committed unchecked and interpreted by application circuits and SDKs.

On owner-signed transaction paths, `ConstrainOutput` requires the output owner's signature only when `data_hash` is nonzero. Consequently, nonempty application data paired with a zero hash does not activate this requirement, and the recipient can receive that data without signing the transaction. The SDK's `with_utxo_data` method also accepts application bytes and a caller-supplied hash without validating their relationship.

Integrations that treat proof acceptance or UTXO commitment verification as validation of the application payload can trust false metadata, select UTXOs that fail application checks, or encounter scanning and spending failures. These consequences depend on applications trusting the payload without checking its hash, which makes this an informational integration concern.

Recommendations:

If the Shielded Pool should guarantee this relationship, extend its proof constraints to recompute `data_hash` from the application data using a defined encoding and hash scheme, reserving zero for absent data.

Otherwise, make the existing trust boundary explicit in SDK APIs and integration documentation: applications must verify the payload `against` data_hash before trusting it, and reject nonempty application data paired with a zero hash before treating it as owner-authorized state.

Customer Response:

Acknowledged in bbdecee.

I-02: Ring deposits can drain SPL accounts controlled by ring_auth

Files:

custom-rings/program/src/instructions/forward.rs

Description:

Shielded pool's `ring_deposit` creates shielded notes without a proof and requires both depositor and ring_config to sign. The `ring_config` account is the ring program's `ring_auth` program-derived address (PDA), which the ring program signs for during a cross-program invocation (CPI).

However, the custom-rings forwarder accepts caller-supplied deposit accounts without rejecting `ring_auth` as the depositor. Its CPI helper grants signer status to every occurrence of `ring_auth`. This allows the following flow:

1. A caller supplies the same `ring_auth` PDA as both `depositor` and `ring_config`, with neither role signed in the outer transaction.
2. The ring program grants signer status to both roles during CPI. Shielded pool accepts the PDA as the depositor and transfers tokens from an SPL account whose authority is `ring_auth` into the pool's token vault.
3. The deposit creates a shielded note for a recipient controlled by the caller.

This allows callers to take funded SPL balances controlled by `ring_auth` when an active, unpaused ring accepts the deposit and permits that depositor assignment. The provided custom-rings forwarder permits the assignment. The finding is informational because its root cause lies in custom ring programs, which are outside the scope of this review.

Recommendations:

Fix the validation in the custom ring program before forwarding the deposit to shielded pool. Reject deposits whose payer or depositor address equals `ring_auth`, and require the depositor to be a signer on entry to the ring program, before the CPI helper grants PDA signer privileges.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. The new custom ring program requires the actual depositor or payer to be a signer before the forwarder adds `ring_auth` privileges.

I-03: Ring authority can withdraw through public settlement

Files:

prover/server/circuits/spp_transaction/custom/authority.go

Description:

The ring-authority circuit permits spending without an input-owner signature and requires every real input and output UTXO to belong to the same ring. However, this restriction applies only to shielded UTXOs. Its shared balance constraints still accept negative public amounts, which represent withdrawals and reduce the value retained in shielded outputs.

If a ring program permits public settlement on this path, an authority transaction can withdraw ring funds without the input owner's signature. The authority must still know the input's nullifier secret, use an enabled authority path, and satisfy the ring program's authorization checks.

This contradicts the specification's claim that ring membership prevents value from leaving through the authority path. The reference policy ring already rejects public settlement on its delegate path, so its documented no-withdrawal property holds through that additional check. Other integrations relying on SPP alone can unintentionally grant the authority withdrawal capability.

Recommendations:

Document that the ring-authority circuit confines shielded UTXOs to the ring but permits public withdrawals. Update the specification and policy documentation to distinguish SPP's guarantees from the reference ring's restrictions, and state that ring programs must reject public withdrawals on the authority path when their intended policy prohibits them.

Customer Response:

Acknowledged in bbdecee.

I-04: Sha256BE returns incorrect zero nodes

Files:

program-libs/hasher/src/sha256.rs

Description:

`Sha256BE` computes SHA-256 and clears the first byte of each result. Its `zero_bytes` method should return predefined hashes of empty Merkle subtrees computed with the same hashing rule. However, it returns the zero-node table for ordinary `Sha256`. The mismatch appears at level 1:

1. Starting with a leaf of 32 zero bytes, `Sha256BE::hashv([zero, zero])` produces a hash beginning with `0x00`.
2. `Sha256BE::zero_bytes()[1]` instead begins with 245 (`0xf5`), as defined in the ordinary SHA-256 table.

No production tree in the reviewed code uses `Sha256BE` as its node hasher. The state tree uses Poseidon, while `Sha256BE` calls in deposit and transact only compute hashes and do not read zero nodes. No current fund-loss or proof-bypass impact was identified. A future Merkle-tree caller using this table would receive empty-subtree values inconsistent with `Sha256BE` hashing.

Recommendations:

Generate a separate `Sha256BE` zero-node table and return it from `Sha256BE::zero_bytes`. Start with 32 zero bytes at level 0, then compute each next level as `Sha256BE::hashv([previous, previous])`. Clearing only the first byte of each existing SHA-256 entry is insufficient, because changing a lower-level value changes the input to every higher-level hash.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. The new code now gives `Sha256BE` its own recursively generated zero-node table. Each level now uses the preceding masked children, producing roots consistent with `Sha256BE` while preserving ordinary `SHA256` behavior.

I-05: P256 UTXOs have limited support outside custom rings

Description:

P256 transfers and withdrawals depend on custom rings. Three related limits affect P256 UTXOs outside these rings.

- A proofless deposit can create a P256 UTXO in the default ring, but the direct transaction path accepts only `ConfidentialEddsa`. Spending this deposit requires a compatible, active, unpaused custom ring that accepts the transaction. Without such a ring, the owner cannot transfer or withdraw the funds.

- A custom-ring P256 spend cannot return funds to the same P256 owner in the default ring. `AssertDefaultP256Owner` forbids publishing the signing P256 identity, while `AssertPublishedOutputOwners` requires default-ring outputs to publish their actual owner identity. These rules conflict when the output belongs to the same P256 key, so proof verification fails.

- A separate identity mismatch blocks default-ring P256 outputs on `RingP256`. The pool uses `solana_owner_identity(x)`, while the output commitment uses `p256_owner_identity(x)`, where `x` is the recipient's P256 public-key x-coordinate. These identities differ, but the circuit requires them to match. This rejects outputs to both the same P256 owner and a different P256 recipient. Transfers within the custom ring, transfers to default-ring Ed25519 recipients, and public withdrawals remain available if the ring permits them.

Recommendations:

The documentation must clearly state that P256 UTXOs are designed to operate within custom rings and explain these limits before users deposit. Accepting a default-ring P256 deposit does not guarantee that its owner can spend it.

Customer Response:

Acknowledged in bbdecee.

I-06: Wrong error variant in UtxoTreeLayout::init

Files:

program-libs/tree/src/smt.rs

Description:

`UtxoTreeLayout::init` converts `ROOT_HISTORY_CAPACITY` from `usize` to `u16` before storing the root history capacity. A failed conversion is mapped to `TreeError::HeightTooLarge`, even though this check concerns the capacity rather than the tree height. The conversion would fail if `ROOT_HISTORY_CAPACITY` exceeded 65,535.

In the reviewed code, `ROOT_HISTORY_CAPACITY` is fixed at 500, and a compile-time assertion prevents it from exceeding `u16::MAX`. The failure branch is therefore unreachable, so this inconsistency has no current runtime or security impact. If future changes made the branch reachable, the error would misidentify a capacity problem as an unsupported tree height, complicating diagnosis.

Recommendations:

Map the failure of `u16::try_from(ROOT_HISTORY_CAPACITY)` to the existing `TreeError::InvalidCapacity` variant instead of `TreeError::HeightTooLarge`.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. The root history capacity conversion failure now emits `InvalidCapacity` instead of `HeightTooLarge`.

I-07: HashChain4 does not bind input length

Files:

prover/server/circuits/gadget/hashchain.go

Description:

`HashChain4` commits to public-input transcripts and nullifier-batch values by starting with the first element and absorbing the remaining elements in groups of three. Each step hashes the current chain value and the group using four-input Poseidon. An incomplete final group is padded with zeros, but the original list length is never included in the commitment.

Consequently, different-length lists have the same digest when explicit trailing zeros replace padding. For example:

- `HashChain4([a, b])` hashes `Poseidon(a, b, 0, 0)`, with both zeros supplied as padding.
- `HashChain4([a, b, 0])` and `HashChain4([a, b, 0, 0])` hash the same four values. Their explicit zeros are indistinguishable from padding. This equality applies while the added zeros fit within the final group; crossing the group boundary introduces another hash operation.

The reviewed transaction and merge paths use fixed transcript layouts and select the corresponding verifying keys, while nullifier-batch circuits enforce their configured batch length. SPP also rejects zero nullifiers and output hashes in published transaction slots. No current proof-bypass or state-integrity impact was identified. A future caller accepting meaningful trailing zeros in variable-length lists could nevertheless mistake distinct lists for the same commitment.

Recommendations:

Keep `HashChain4` restricted to callers that enforce a fixed input length or otherwise exclude this padding ambiguity. If variable-length commitments are required, include an explicit length and domain tag in the hashed preimage.

Customer Response:

Acknowledged in bbdecee.

I-08: Nullifier PDAs lack a type discriminator

Files:

program-libs/interface/src/state/nullifier_pda.rs

Description:

NullifierPda stores `queue_index` and `tree_id` in a fixed 10-byte record without an explicit account type discriminator. `close_nullifier_pdas` uses `load_nullifier_pda` to identify records by program ownership, writability, exact length, nonzero `queue_index`, and matching `tree_id`. It then checks `queue_index` against `close_before_index` without re-deriving the account address.

This makes account typing depend on an implicit invariant: no other initialized account owned by the program has a compatible 10-byte layout. If a future account type satisfies the same field checks, the authorized forester could unintentionally submit it for closure. The instruction would erase its state and transfer its lamports to the tree.

No conflicting account type was found in the reviewed code. Creation verifies the canonical nullifier PDA, externally assigned zero records are rejected, and closure requires the configured forester's signature. Consequently, no current exploit is established. This is a maintenance risk that could become a state-loss vulnerability after future changes.

Recommendations:

Add a unique discriminator to the nullifier record, set it during creation, and require it in `load_nullifier_pda`.

Customer Response:

Fixed in bbdecee.

Fix Review:

Fix confirmed. Nullifier PDAs now have a discriminator and enforce the expected layout.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.